

Lauda Local

Front-end

Como a interface foi construída
e como escrever um prompt para mudá-la.

Versão 0.10.0-beta | 23/09/2026

Documento para quem vai alterar a tela

Sumário

1. O que é o front-end deste projeto	2
Os três arquivos que formam a interface	2
A árvore da janela	2
2. A camada de tema	3
O que mais mora em theme.py	4
3. Os widgets arredondados	5
Como as bordas ficam lisas	5
Catálogo	5
Os dois contratos que todo widget novo precisa honrar	6
4. A janela	7
As coleções de repintura	7
A ponte entre o processamento e a tela	7
5. O que trava mudanças	8
6. Como montar um prompt para alterar o front-end	9
6.1 O bloco de contexto para colar	10
6.2 O vocabulário que faz diferença	11
6.3 Receitas por tipo de mudança	12
6.4 Três pedidos, do vago ao acionável	13
6.5 Padrões de pedido que costumam dar errado	14
6.6 Antes de enviar e depois de receber	14
7. Anexo: mapa rápido	15
Onde fica o quê	15
Comandos	15

1. O que é o front-end deste projeto

A interface do Lauda Local é uma **janela nativa do Windows**, desenhada com Tkinter. Não existe navegador, não existe servidor local, não existe HTML, CSS ou JavaScript em lugar nenhum. Isso foi um requisito explícito: o aplicativo tinha de abrir uma janela própria, não uma página web em localhost.

A consequência prática é que **tudo o que você veria como CSS aqui é código Python**. Não há folha de estilo para editar: cores moram em um dataclass, formas são desenhadas em um Canvas, e o layout é declarado com os gerenciadores de geometria do Tk (grid e pack). Quem for pedir uma mudança precisa falar nesses termos — é disso que trata a segunda metade deste documento.

Os três arquivos que formam a interface

Arquivo	Linhas	Responsabilidade
<code>theme.py</code>	215	As duas paletas (clara e escura), a detecção do tema do Windows e a preferência salva do usuário. Nenhuma cor deve existir fora daqui.
<code>widgets.py</code>	1750	Os widgets arredondados desenhados em Canvas: cartão, botão, navegação lateral, zona de soltar, interruptor, trilha de etapas, barra de progresso, controle deslizante, barra de rolagem, moldura de campo — e os ícones de traço.
<code>desktop.py</code>	1480	A janela: monta o layout, aplica o tema, lê o que o usuário escolheu, dispara o processamento em outra thread e mostra o resultado.

Só esses três

Qualquer mudança visual acontece dentro deles. Se um pedido de alteração de tela levar você a mexer em **pipeline.py**, **runner.py** ou **report.py**, o pedido provavelmente misturou aparência com comportamento — vale separar em dois.

A árvore da janela

É útil ter esse desenho na cabeça antes de pedir qualquer coisa, porque quase todo pedido é, no fundo, “mexa neste galho”:

```
root (Tk)
+-- outer
  +-- nav (SideNav, 158 px) 7 paginas + versao e estado no pe
  +-- content mostra a pagina escolhida na barra lateral
    +-- tab_job Novo trabalho
      | +-- cartao esquerdo DropZone + linha do arquivo
      | +-- cartao direito idioma, qualidade, recursos, saida
      | +-- rodape botao Processar + Stepper + progresso
    +-- tab_report Relatorio (+ abrir pasta / abrir / copiar)
    +-- tab_plain Transcricao
    +-- tab_files Arquivos (ultimo trabalho + historico)
    +-- tab_limits Desempenho (Resumo/Limites/Diagnostico)
    +-- tab_help Ajuda (o passo a passo)
    +-- tab_settings Configuracoes (tema, preferencias, Ollama)
```

2. A camada de tema

Theme é um dataclass congelado com 21 cores nomeadas **pela função, não pela cor**. Existem duas instâncias prontas: LIGHT e DARK. Trocar de tema é trocar a instância e repintar — não há um segundo conjunto de estilos.

Token	Para que serve	Claro	Escuro
<code>canvas</code>	Fundo da janela	#F4F6FA	#0F1218
<code>paper</code>	Fundo dos painéis e cartões	#FFFFFF	#171B23
<code>surface</code>	Áreas internas (zona de soltar)	#F7F9FC	#1C222C
<code>sidebar</code>	Fundo da barra lateral	#EDF0F5	#12161D
<code>nav_active</code>	Item de navegação atual	#FFFFFF	#232A36
<code>field</code>	Fundo de campos de entrada	#FFFFFF	#1F242E
<code>ink</code>	Texto principal	#12151C	#E8EAF0
<code>ink_soft</code>	Texto secundário e dicas	#4A5163	#9AA3B4
<code>line</code>	Bordas e divisórias	#D9DEE7	#2B313D
<code>muted</code>	Interruptor desligado, etapa pendente	#BFC7D4	#3B4351
<code>accent</code>	Títulos de passo, aba ativa	#1F7A73	#5ECDC3
<code>accent_warm</code>	Avisos	#B26A00	#FFB02E
<code>danger</code>	Erros	#B3261E	#FF6B60
<code>success</code>	Conclusão	#1F7A73	#5ECDC3
<code>button / _active / _text</code>	Botão comum	#EDF0F5	#242A35
<code>primary / _active / _text</code>	Botão Processar	#1F7A73	#2A8C84
<code>preview_bg / preview_fg</code>	Área de pré-visualização	#FFFFFF	#12151C
<code>selection / selection_text</code>	Texto selecionado	#CDE7E4	#2A4C4A

O que mais mora em **theme.py**

- `detect_system_theme()` — lê a chave `AppsUseLightTheme` do registro do Windows para o modo **auto**.
- `load_choice()` / `save_choice()` — a preferência vai para `~/.lauda/ui.json`, junto dos limites de máquina. Gravação preserva as outras chaves do arquivo.
- `apply_titlebar()` — pinta a barra de título de escuro via `DwmSetWindowAttribute`. É a única parte da janela que o Tk não desenha.

A regra que sustenta o modo escuro

Nenhum arquivo além de **theme.py** pode conter um código hexadecimal de cor. Se um widget novo precisar de um tom que não existe na paleta, o caminho é criar um token novo nos dois temas — nunca escrever a cor no lugar onde ela é usada. Um teste verifica que as duas paletas têm exatamente os mesmos campos.

3. Os widgets arredondados

O ttk não desenha cantos arredondados. Não é questão de configuração: o desenho de cada widget vem do tema nativo, e nenhum dos temas disponíveis arredonda nada. Para conseguir o visual pedido, os widgets foram **redesenhados sobre** `tk.Canvas`, que aceita qualquer forma.

Como as bordas ficam lisas

O Canvas do Tk não faz antialiasing — um canto arredondado feito de polígono sai serrilhado. A solução foi desenhar a superfície como imagem com o **Pillow em 4x** e reduzir: o resultado fica liso. Sem Pillow instalado, o código cai para o polígono suavizado do próprio Tk, mais feio mas funcional. Nenhuma dependência nova virou obrigatória por causa disso.

Catálogo

Classe	Substitui	O que ganha	API relevante
<code>RoundedSurface</code>	—	Base de todos: desenha a superfície e se repinta ao redimensionar	<code>set_surface()</code> , <code>redraw()</code>
<code>RoundedCard</code>	<code>ttk.Frame</code>	Painel com canto redondo e borda fina; o conteúdo vai em <code>.body</code>	<code>apply_theme()</code>
<code>RoundedButton</code>	<code>ttk.Button</code>	Botão com hover, estado pressionado e variante principal	<code>configure(state=)</code> , <code>cget()</code> , <code>invoke</code> por clique
<code>RoundedProgress</code>	<code>ttk.Progressbar</code>	Barra com trilha e preenchimento arredondados	<code>configure(value=, maximum=)</code>
<code>NavItem / SideNav</code>	<code>ttk.Notebook</code>	Barra lateral com ícone e rótulo; troca a página mostrada	<code>add(frame, text, icon)</code> , <code>select(frame)</code>
<code>DropZone</code>	—	Área de soltar/escolher arquivo, com moldura tracejada e nuvem	<code>command</code> , <code>set_active()</code> , <code>set_texts()</code>
<code>ToggleSwitch</code>	<code>ttk.Checkbutton</code>	Interruptor deslizante com o rótulo à esquerda	<code>variable</code> , <code>text</code> , <code>command</code>
<code>Stepper</code>	—	Trilha das cinco etapas do processamento, no rodapé	<code>set_current(index)</code> , <code>current</code>
<code>tinted_icon</code>	—	Carrega um PNG e o repinta com a cor do tema; o arquivo entra só como forma (é a nuvem da zona de soltar)	<code>tinted_icon(caminho, tam, cor)</code>
<code>draw_icon</code>	—	Os ícones de traço (nuvem, pasta, engrenagem, play...) desenhados em linhas, para acompanharem a cor do tema	<code>draw_icon(canvas, nome, x, y, tam, cor)</code>
<code>RoundedSlider</code>	<code>ttk.Scale</code>	Trilha arredondada com botão circular	<code>get()</code> , <code>set()</code> , <code>command</code>
<code>RoundedField</code>	—	Moldura arredondada para hospedar <code>Entry</code> e <code>Combobox</code> do ttk, que ficam sem borda própria	<code>attach(widget)</code>
<code>RoundedScrollbar</code>	<code>ttk.Scrollbar</code>	Barra fina, sem setas, com cursor em pílula; some quando não há o que rolar	<code>set(first, last)</code> e “moveto” no <code>command</code>

Os dois contratos que todo widget novo precisa honrar

1. Um método `apply_theme(...)`. Como o widget é pintado no Canvas, o ttk não alcança as cores dele: quem repinta é a própria janela, chamando esse método na troca de tema. Sem ele, o widget fica com a cor do tema anterior — o defeito clássico dessa abordagem.

2. A fatia da API do ttk que o aplicativo usa. `RoundedButton` aceita `configure(state=...)`, `cget()` e indexação; `RoundedScrollbar` fala o protocolo de rolagem do Tk. Isso é o que permitiu trocar os widgets sem reescrever quem os usa — nem os testes.

```
def apply_theme(self, *, fill, outline, parent_bg) -> None:
    self.set_surface(fill=fill, outline=outline, parent_bg=parent_bg)

# parent_bg existe porque o Canvas não é transparente: o widget precisa
# saber a cor de quem está atrás dele para o canto redondo não aparecer
# recortado sobre um retângulo de cor errada.
```

4. A janela

LaudaApp concentra a montagem e o estado da tela. A ordem de construção importa e é sempre a mesma:

```
__init__
+-- colecoes de repintura vazias  (_cards, _buttons, _checks, ...)
+-- carrega tema e limites salvos (~/.lauda/ui.json)
+-- _build_window  titulo, tamanho 1200x800, icone
+-- _build_fonts   Segoe UI 9/10/11/21 e Consolas 10
+-- _build_layout  header + cartao esquerdo + cartao direito
+-- _apply_theme   pinta tudo pela primeira vez
```

As coleções de repintura

Este é o detalhe mais importante para quem vai mexer na tela. Como os widgets arredondados são invisíveis para o ttk, a janela guarda listas de tudo o que precisa repintar à mão:

```
self._cards      RoundedCard      self._switches  interruptores
self._buttons    RoundedButton    self._scrollbars barras de rolagem
self._fields     Entry e Combobox  self._texts     areas de previa
self._dividers   linhas divisorias self._primary_buttons cor primaria
self._panels / self._shells / self._sidebar_panels frames, por cor

Fora das listas, repintados um a um: self.nav, self.stepper e self.drop.
```

A regra de ouro ao adicionar um widget

Todo widget desenhado em Canvas que entrar na tela precisa ser **acrescentado à coleção correspondente** e repintado dentro de **_apply_theme**. Esquecer disso não quebra nenhum teste hoje — só aparece quando alguém troca de tema e um pedaço da tela fica com a cor errada. É o erro mais fácil de cometer neste código.

A ponte entre o processamento e a tela

O Tk é de thread única: só a thread da interface pode tocar em um widget. O processamento roda em outra thread (e, por baixo, em outro processo), então a comunicação passa por uma fila:

```
worker (thread)  ---> queue.Queue  ---> _drain_queue (a cada 80 ms)
                                     +-- _on_progress
                                     +-- _on_recovery
                                     +-- _on_done / _on_error
```

Todo pedido que envolva mostrar algo **durante** o processamento passa por aqui: a mensagem entra na fila e o desenho acontece do lado da interface. Tocar num widget direto da thread de trabalho trava a janela de forma intermitente.

5. O que trava mudanças

São as regras que o código já segue. Um prompt que não as mencione tende a produzir uma alteração que funciona na tela e quebra em outro lugar.

Regra	Por quê	Como é verificada
Interface, logs e textos em português do Brasil; identificadores de código em inglês	Foi definido no início do projeto e vale para tudo	Revisão; o texto da tela está todo em desktop.py
Nenhuma cor fora de theme.py	É o que faz o modo escuro funcionar por inteiro	test_paletas_tem_os_mesmos_campos, test_todas_as_cores_sao_hexadecimais_validas
Nenhum widget retangular do ttk na tela	Checkbox, Scrollbar, Notebook, Scale e Progressbar do ttk não arredondam	test_nenhum_widget_retangular_do_ttk_sobrou_varre_a_arvore_da_janela
Todo widget em Canvas entra numa coleção e é repintado	Senão fica com a cor do tema anterior	test_alterna_para_o_modos_escuro, test_o_modos_escuro_repinta_a_previa
A troca de tema não pode apagar o que está na tela	O usuário troca de tema depois de processar	test_troca_de_tema_nao_apaga_o_relatorio
ruff e mypy limpos	É a barra de qualidade do projeto inteiro	ruff check src tests scripts; mypy src

Os testes de interface não abrem janela

A raiz é criada e escondida com **withdraw()**, e o laço de eventos é bombeado à mão. Isso tem uma consequência: o Tk não entrega eventos de mouse a widget não mapeado, e a geometria devolve 1x1. Por isso os testes de clique chamam o tratador com um evento montado à mão, depois de verificar que o vínculo existe. Vale saber disso antes de pedir “um teste que clique no botão”.

6. Como montar um prompt para alterar o front-end

Um bom prompt de front-end aqui responde cinco perguntas. Faltando qualquer uma delas, o resultado costuma ser uma mudança que *parece* certa na captura de tela e erra em outro estado — no tema oposto, com a janela menor, durante o processamento, ou no arquivo de preferências.

Parte	O que dizer	Exemplo de frase
1. Alvo	Onde na tela, com o nome que o código usa	"no cartão da esquerda, no passo 3"
2. Comportamento	O que deve acontecer, incluindo os estados	"desabilitado enquanto processa; volta ao normal ao terminar"
3. Restrições	As regras do projeto que valem para esse pedido	"usando token do theme.py, sem widget retangular do ttk"
4. Prova	Como você quer confirmar que funcionou	"teste novo e captura nos dois temas"
5. Escopo	O que não é para mexer	"não mude o pipeline nem o formato do relatório"

6.1 O bloco de contexto para colar

Quando a conversa começa do zero — sessão nova, outra ferramenta, outra pessoa — cole isto antes do pedido. Ele economiza a rodada de perguntas e evita as três suposições erradas mais comuns (que é web, que dá para usar ttk normalmente, e que cor se escreve onde é usada):

Contexto do projeto (Lauda Local, front-end):

- Aplicativo de janela nativa em Python + Tkinter. Não é web: não existe HTML, CSS, JS, navegador nem servidor local. Não proponha nenhum.
- A interface vive em 3 arquivos, dentro de `src/lauda/`:
 - `theme.py` paletas clara e escura (21 tokens de cor) + preferencia
 - `widgets.py` widgets arredondados desenhados em `tk.Canvas`
 - `desktop.py` a janela: layout, tema, eventos, resultado
- A tela é uma barra lateral (SideNav) com 6 páginas. A primeira, 'Novo trabalho', tem a zona de soltar (DropZone), o painel de recursos com interruptores (ToggleSwitch) e o rodapé com o botão Processar mais a trilha de etapas (Stepper).
- O ttk não arredonda canto, então Card, Button, Nav, Progress, Slider, Switch e Scrollbar são classes próprias em `widgets.py`. Não volte a usar `ttk.Checkbutton`, `ttk.Scrollbar`, `ttk.Notebook`, `ttk.Scale` nem `ttk.Progressbar`: existe um teste que falha se algum aparecer na tela.
- Nenhuma cor hexadecimal fora de `theme.py`. Precisando de um tom novo, crie um token nos dois temas.
- Todo widget desenhado em Canvas precisa de `apply_theme()` e precisa ser adicionado a coleção correspondente (`self._cards`, `self._buttons`, `self._switches`, `self._fields`, `self._scrollbars`, ...) para ser repintado em `_apply_theme`.
- Textos da interface em português do Brasil; identificadores em inglês.
- Nenhum PNG colorido na interface: ícone entra como forma e recebe a cor do tema (`widgets.tinted_icon`), senão fica errado num dos dois modos.
- O Tk é de thread única: o que vem do processamento chega pela fila e é desenhado em `_drain_queue`, nunca direto da thread de trabalho.
- Barra de qualidade: ruff check src tests scripts e mypy src limpos, e pytest tests passando.

6.2 O vocabulário que faz diferença

Pedidos de interface falham por ambiguidade de nome mais do que por ambiguidade de intenção. À esquerda, como a vontade costuma sair; à direita, o termo que o código entende.

Se você quer dizer	Diga assim	Onde isso mexe
"o menu do lado"	a barra lateral (SideNav)	desktop.py, _build_layout
"a área pontilhada"	a zona de soltar (DropZone)	desktop.py, _build_file_card
"as chavinhas"	os interruptores (ToggleSwitch) do painel Recursos	desktop.py, _build_options_card
"as bolinhas do rodapé"	a trilha de etapas (Stepper)	desktop.py, _build_action_bar
"a bolinha que arrasta"	o controle deslizante (RoundedSlider) da aba Desempenho	desktop.py, _build_limits_tab
"a caixa de texto grande"	a pré-visualização (tk.Text de _text_tab)	desktop.py, _text_tab
"a cor do fundo"	o token canvas (janela) ou paper (painéis)	theme.py
"a cor do texto apagado"	o token ink_soft	theme.py
"mais arredondado"	aumentar o radius do widget	widgets.py
"mais espaçado"	aumentar padding/pady no grid	desktop.py
"o botão verde"	o botão principal (primary), o Processar	desktop.py + theme.py

6.3 Receitas por tipo de mudança

O que citar no prompt, de acordo com o que se quer. A coluna da direita é o que evita a resposta incompleta.

Você quer	Arquivos	Não esqueça de pedir
Mudar uma cor	theme.py	o valor nos dois temas, claro e escuro
Mudar o arredondamento ou a espessura de um widget	widgets.py	que o padrão continue valendo para todas as instâncias, ou que vire parâmetro
Acrescentar um campo, botão ou opção	desktop.py	entrar na coleção de repintura, aparecer em <code>_apply_theme</code> e, se for opção de processamento, chegar em <code>JobOptions</code>
Reorganizar o layout	desktop.py	que continue legível em 1200x800 e que nada seja cortado — a coluna esquerda já ficou apertada uma vez
Criar um widget arredondado novo	widgets.py + desktop.py	<code>apply_theme()</code> , coleção, teste, e a fatia da API do <code>ttk</code> que ele substitui
Mudar um texto da tela	desktop.py	conferir se o mesmo texto aparece no README ou nos PDFs
Acrescentar uma página	desktop.py	<code>nav.add(frame, texto, ícone)</code> e o fundo do frame no token <code>paper</code>
Mudar o comportamento em tempo de processamento	desktop.py	passar pela fila e por <code>_drain_queue</code> , nunca direto da <code>thread</code>

6.4 Três pedidos, do vago ao acionável

O mesmo desejo, escrito de três formas. A diferença entre eles é exatamente o que sobra de trabalho de adivinhação.

Vago

```
deixa a tela mais bonita
```

Não diz onde, não diz o que incomoda, não diz como conferir. A resposta provável é uma mudança grande em lugar nenhum específico — e é preciso desfazer boa parte dela.

Melhor

```
deixa os cantos mais arredondados na parte de dentro do painel direito
```

Já dá para agir: identifica a região. Ainda falta dizer quais elementos incomodam e como saber que terminou.

Acionável

```
No cartao da direita, as caixas de selecao e a barra de rolagem ainda aparecem quadradas.
```

```
Troque as duas por widgets desenhados em Canvas, no mesmo estilo dos que ja existem em widgets.py:
```

- a caixinha com canto arredondado e um 'v', no lugar do quadrado com 'x' do tema clam;
- a barra de rolagem fina, sem setas, cursor em pilula, sumindo quando nao ha o que rolar.

```
Restricoes: cores so por token do theme.py, nos dois temas; as duas classes precisam de apply_theme() e de entrar nas colecoes de repintura; a barra de rolagem tem de falar o protocolo do Tk para entrar no lugar da ttk.Scrollbar sem mudar o tk.Text.
```

```
Prova: um teste para cada comportamento novo, mais um teste que falhe se algum widget retangular do ttk voltar para a arvore da janela. Rode ruff, mypy e a suite inteira, e me mostre uma captura nos dois temas.
```

```
Nao mexa no pipeline nem no formato do relatorio.
```

Esse último é, quase literalmente, o pedido que originou o trabalho descrito neste documento. Ele cabe em vinte linhas e não deixa decisão importante em aberto.

6.5 Padrões de pedido que costumam dar errado

Pedido	O que acontece	Como reescrever
"usa a cor #2E86AB no botão"	A cor entra escrita no desktop.py e o modo escuro fica com um botão fora da paleta	"crie um token para essa cor no theme.py, com uma variante escura, e use no botão"
"coloca um Notebook com mais abas"	Volta o widget retangular do ttk e um teste falha	"acrescente abas ao RoundedTabs que já existe"
"atualiza a barra de progresso dentro do processamento"	Toca em widget de outra thread; a janela trava de forma intermitente	"emita um evento de progresso na fila e desenhe em _drain_queue"
"faz igual ao aplicativo X"	Sem referência concreta, cada elemento é reinterpretado	Descreva o elemento e o comportamento; se tiver imagem, anexe e diga o que dela importa
"arruma o CSS"	Não existe CSS; o tempo vai embora explicando isso	"ajuste o estilo em theme.py / widgets.py"
"deixa responsivo"	Termo de web; aqui não há breakpoint	"garanta que nada seja cortado com a janela em 1200x800 e ao maximizar"

6.6 Antes de enviar e depois de receber

No pedido

- Nomeei o lugar com o nome que o código usa?
- Descrevi o comportamento nos estados — normal, desabilitado, processando, erro?
- Disse que vale nos **dois** temas?
- Pedi teste e captura de tela?
- Disse o que **não** é para mexer?

Na entrega

- A suíte inteira passou, não só os testes de interface.
- `ruff check src tests scripts` e `mypy src` limpos.
- Nenhum hexadecimal novo fora do **theme.py**.
- Alternar o tema com a tela cheia de conteúdo não deixa nada com a cor antiga nem apaga o relatório.
- A janela em 1200x800 não corta o botão Processar.
- As capturas em **docs/** ainda correspondem ao que a tela mostra.

7. Anexo: mapa rápido

Onde fica o quê

Quero mexer em	Vá para
A barra lateral e as páginas	desktop.py — <code>_build_layout</code>
Diagnóstico da máquina	hardware.py — <code>assess_machine</code>
O aviso de máquina fraca	desktop.py — <code>show_machine_dialog</code>
A zona de soltar e o arquivo	desktop.py — <code>_build_file_card</code>
Recursos, idioma, qualidade, pasta	desktop.py — <code>_build_options_card</code>
Processar e a trilha de etapas	desktop.py — <code>_build_action_bar</code>
Idioma e qualidade (as listas)	desktop.py — <code>LANGUAGES</code> , <code>MODEL_LABELS</code>
Texto de “Como usar”	desktop.py — <code>STEPS_TEXT</code>
Páginas de texto e pré-visualização	desktop.py — <code>_text_page</code>
Sliders de CPU, RAM, GPU e VRAM	desktop.py — <code>_build_limits_page</code>
Nomes das etapas	desktop.py — <code>STAGE_LABELS</code> , <code>PIPELINE_STEPS</code>
Fontes	desktop.py — <code>_build_fonts</code>
Toda a repintura de tema	desktop.py — <code>_apply_theme</code>
Formas e desenho	widgets.py
Cores	theme.py — <code>LIGHT</code> e <code>DARK</code>

Comandos

```
python -m lauda.desktop      abre a janela
pytest tests/test_desktop.py -q  so os testes de interface
pytest tests -q              a suite inteira
ruff check src tests scripts  estilo
mypy src                     tipos
python scripts/make_frontend.py  regenera este PDF
```

Os outros documentos

Como usar e pra que serve é o guia do usuário; **Manual** cobre a operação em detalhe; **Decisões técnicas** explica o porquê da arquitetura inteira, incluindo transcrição, diarização e desempenho. Este aqui trata só da tela.